COMP4620/8620 Advanced Topics in AI: Intelligent Robotics Assignment 2 Report

Hadis Amin - u8050066

Part A

A1. State abstraction of the 1-D grid MDP

No. Abstracting the state into L and R does not turn this into an MDP. An MDP assumes the agent knows exactly which state it is in. Here the true state is still uncertain: we only believe the agent is somewhere on the left side or the right side, not exactly where. The agent therefore still needs a belief, a probability distribution over the possible states, so the problem remains a POMDP.

A2. α-vector policy representation

Let $b = (p, 1-p)$ where $p = b(s_1) \in [0, 1]$. Then $\alpha \cdot b = \alpha(s_1)\, p + \alpha(s_2)\,(1-p)$.

α	vector	$V_\alpha(p) = \alpha \cdot b$
α_1	$[10, -5]$	$15p - 5$
α_2	$[-5, 10]$	$-15p + 10$
α_3	$[4, 4]$	4
α_4	$[1, 2]$	$-p + 2$
α_5	$[5.5, 1]$	$4.5p + 1$

Table 1: Value of each α-vector as a function of $p = b(s_1)$.

(a) Belief intervals where each α-vector is maximal. Figure 1 plots the five lines over $p \in [0, 1]$. The shaded upper surface is made of three segments: α_2 on the left, α_3 in the middle and α_1 on the right. The breakpoints (marked in the figure) are the intersections $\alpha_2 = \alpha_3 \Rightarrow -15p + 10 = 4 \Rightarrow p = 0.4$ and $\alpha_3 = \alpha_1 \Rightarrow 15p - 5 = 4 \Rightarrow p = 0.6$. The figure also shows that α_4 and α_5 lie strictly below the surface everywhere: α_4 never exceeds $2 < 4$, and α_5 is below α_3 on $[0, 2/3)$ and below α_1 for $p > 4/7$, so neither is ever maximal. The resulting intervals are summarised in Table 2.

α	Interval of $p = b(s_1)$ where maximal
α_1	[0.6, 1]
α_2	[0, 0.4]
α_3	[0.4, 0.6]
α_4	never
α_5	never

Table 2: Belief intervals where each α-vector is maximal (read off Figure 1).

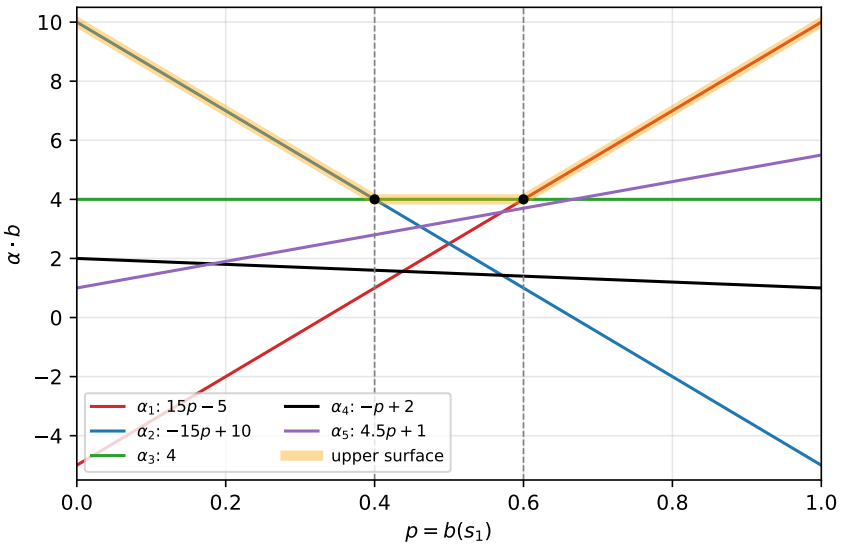


Figure 1: The five α-vectors over $p \in [0, 1]$ and the upper surface of Γ (shaded), with breakpoints at $p = 0.4$ and $p = 0.6$.

(b) Pruning by pointwise dominance. From Figure 1, α_4 and α_5 lie strictly below the maximum for every belief $p \in [0, 1]$, so both can be removed by pointwise dominance. The maximum is

$$V_{\max}(p) = \begin{cases} V_{\alpha_2}(p) = 10 - 15p, & 0 \le p \le 0.4, \\ V_{\alpha_3}(p) = 4, & 0.4 < p < 0.6, \\ V_{\alpha_1}(p) = 15p - 5, & 0.6 \le p \le 1. \end{cases}$$

For all $p \in [0, 1]$, $V_{\alpha_4}(p) = 2 - p < V_{\max}(p)$ and $V_{\alpha_5}(p) = 4.5p + 1 < V_{\max}(p)$. The dominating vector may differ with p. In contrast, α_1, α_2 and α_3 each attain the maximum on an interval, so they cannot be removed.

A3. Pedestrian avoidance: MDP vs POMDP

Scenario where MDP is suitable.

- S: [car_position, pedestrian_position]
- A: [Stay, Move]

Sensors accurately identify both positions. Assume movement is discrete and the next state depends only on the current positions and action. An MDP policy maps the observed state to the best action: if a pedestrian blocks the cell ahead, choose Stay; otherwise, choose Move when safe. Since the complete state is observed, a POMDP belief is unnecessary.

Scenario where POMDP is suitable. **[TODO: Describe the scenario, e.g. what part of the state the car cannot observe even with good sensors (pedestrian intention/destination?).]**

- State (observed and hidden parts): **[TODO:]**
- Observations: **[TODO:]**
- Why reasoning over a belief changes the decision: **[TODO:]**

Part B

B1. MDP framing of multi-drone navigation

- **State space** S: each state contains the positions of all N drones, $s = (p_1, \ldots, p_N)$, where $p_i = (x_i, y_i, z_i)$.

- **Action space** A: A joint action $a = (a_1, \ldots, a_N)$ specifies the simultaneous action of every drone. Thus, $A = A_1 \times \cdots \times A_N$ and $|A| = 26^N$. For example, two drones have $26^2 = 676$ joint actions.
- **Transition function** $T(s, a, s') = T_{known}$
- **Reward function** for N drones is the sum over all drones:

$$R(s, a, s') = \sum_{i=1}^{N} R_i(s, a, s'),$$

where

$$R_i(s, a, s') = -1 + \begin{cases} -50, & \text{if drone } i \text{ collides,} \\ 100, & \text{if drone } i \text{ reaches its goal.} \end{cases}$$

- **Discount factor** γ: **[TODO: e.g. 0.99]**

B2. Approximate on-line MDP solver

Overview. We use Monte Carlo Tree Search (MCTS) with UCT selection and a greedy rollout policy. At each decision step, the planner builds a search tree rooted at the current joint state within the available planning time (at most 10 seconds, without parallelisation). It estimates action values through sampled transitions and discounted returns, then executes a tried root action with the highest estimated value, breaking equal-value ties uniformly at random. Sampling avoids enumerating all possible successor states, although a large joint action space remains a limitation.

1. Selection

At the current node, untried joint actions are sampled randomly and checked for valid intended moves. Once no eligible untried actions remain, select the tried eligible action with the highest Upper Confidence Bound (UCB):

$$a \leftarrow \arg\max_a \left[Q(\text{node}, a) + \sqrt{\frac{2 \ln N(\text{node})}{N(\text{node}, a)}} \right].$$

Here $Q(\text{node}, a)$ is the estimated mean discounted return, $N(\text{node}, a)$ is the number of times action a has been tried at node, and $N(\text{node}) = \sum_a N(\text{node}, a)$. UCB balances actions with high estimated returns against actions that have been explored less often.

Validity checks and fallback. SAFE(s, a) decodes the joint action and calculates each drone's intended target directly; a drone already at its goal keeps its current position. It returns true only if every target is inside the grid, outside obstacles and distinct from every other target. It does not call `env.simulate`, so stochastic outcomes can still collide. Invalid actions are cached at the node. Selection tests at most 64 randomly chosen untried candidates per call, then uses its cached greedy fallback if none passes or the deadline expires. The fallback is checked when first computed; if it is unsafe because no valid joint assignment exists, it is still returned. Tried actions considered by UCB have already passed the intended-target checks.

2. Expansion

After selecting a joint action a_i at node, call `env.simulate` to sample a successor joint state s_j, reward and terminal flag. Check whether the pair (a_i, s_j) already exists among the children of node:

- **New pair:** Create a child node for s_j, initialise its action visit counts and values to zero, and store it under (a_i, s_j).

- **Existing pair:** Reuse the stored child, move the search to that node and perform Selection again from there.

The parent is the node currently being visited; it is the root at the start of a search iteration. Because motion is stochastic, selecting the same action can produce different successor states, each stored as a separate outcome child. Tree descent continues through existing children until a new child is created, a terminal outcome is sampled, the depth limit `max_num_steps` is reached, or the planning deadline expires.

3. Rollout from a new node

If the newly created child is nonterminal, start a rollout from its state using the greedy policy. Its simulated states are not added to the search tree. If the sampled outcome is terminal, skip the rollout. The total number of transitions in the tree and rollout is limited to $D = \texttt{max_num_steps}$: a child at depth d has at most $D - d$ rollout steps remaining. The planning deadline also applies throughout.

Choosing a greedy action. At each rollout step, process each drone that has not yet reached its goal:

1. For every available per-drone action vector u, calculate the intended candidate position $p_i + u$ directly. There are 26 vectors when altitude changes are enabled and 8 otherwise.
2. Discard candidates outside the grid or inside an obstacle.
3. Measure the Euclidean distance from each remaining candidate to that drone's goal g_i:

$$\text{distance}_i(u) = \|p_i + u - g_i\|_2.$$

4. Order candidates from the shortest distance to the longest. Assign targets in drone order, trying the nearest candidate first. If a target is already assigned to another drone, try the next candidate; backtrack to an earlier drone's choice if necessary to obtain distinct targets for all drones.

Drones that have reached their goals remain in place and occupy their goal cells. If no joint assignment with distinct, valid targets exists, the implementation uses its per-drone fallback choices, which are not guaranteed to avoid collision.

Candidate distances are calculated without calling `env.simulate` for every action. Only after choosing and encoding the joint action does the rollout call `env.simulate` once to sample the next state and reward. Because movement is stochastic, the sampled positions can differ from the intended candidates, so these checks cannot guarantee collision-free outcomes. Repeat greedy action selection from the sampled state until a terminal outcome, the remaining depth limit or the planning deadline is reached, accumulating discounted rewards.

Pseudocode so far. Algorithm 1 shows one search iteration through the rollout call. Algorithm 2 details SELECT, including validity checks, bounded sampling, fallback and UCB; Algorithm 3 gives the rollout loop. GREEDYACTION follows the candidate-distance procedure above. Backpropagation is left for the next discussion.

Algorithm 1: One MCTS iteration: Selection, Expansion and rollout entry

Input: Root node, depth limit $D =$ `max_num_steps`, planning deadline

node $\leftarrow$ root; $d \leftarrow 0$; path $\leftarrow [\,]$; $G \leftarrow 0$;
while *$d < D$ and time remains* **do**
  $a_i \leftarrow$ SELECT(node, deadline);
  if *deadline reached* **then**
    break;
  $(s_j, r, \text{done}) \leftarrow$ `env.simulate`(node.state, a_i);
  Append (node, a_i, r) to path; $d \leftarrow d + 1$;
  $k \leftarrow (a_i, s_j)$;
  isNew $\leftarrow (k \notin$ node.children);
  if *isNew* **then**
    child $\leftarrow$ NEWNODE(s_j) with all $N = 0$, $Q = 0$;
    node.children[k] $\leftarrow$ child;
  child $\leftarrow$ node.children[k];
  if *done* **then**
    break ; `// terminal outcome: no rollout`
  if *isNew* **then**
    $G \leftarrow$ ROLLOUT(child.state, $D - d$, deadline);
    break;
  node $\leftarrow$ child ; `// existing pair: repeat Selection here`

Algorithm 2: SELECT(node, deadline): choose a joint action

Input: Node node with state, visit counts N, values Q and invalid-action flags

if *fallback has not been initialised at node* **then**
  node.fallback $\leftarrow$ GREEDYACTION(node.state);
  node.hasSafe $\leftarrow$ SAFE(node.state, node.fallback);
if *not node.hasSafe* **then**
  return *node.fallback*;
$U \leftarrow \{a : N(\text{node}, a) = 0$ and a is not flagged invalid$\}$;
if $U \neq \varnothing$ **then**
  $m \leftarrow \min(64, |U|)$;
  for $j \leftarrow 1$ **to** m **do**
    if *deadline reached* **then**
      return *node.fallback*;
    Sample a uniformly at random from U;
    if SAFE(*node.state*, a) **then**
      return a;
    Flag a invalid at node; remove a from U;
  return *node.fallback*;
$T \leftarrow \{a : N(\text{node}, a) > 0$ and a is not flagged invalid$\}$;
if $T = \varnothing$ **then**
  return *node.fallback*;
$n \leftarrow \sum_{b \in T} N(\text{node}, b)$;
return $\arg\max_{a \in T} \left[Q(\mathit{node}, a) + \sqrt{\dfrac{2 \ln n}{N(\mathit{node}, a)}} \right]$;

Algorithm 3: ROLLOUT: discounted return using greedy actions

Input: New child state s, remaining depth $L = D - d$, planning deadline

$\hat{s} \leftarrow \text{copy}(s)$; $G \leftarrow 0$; $w \leftarrow 1$;
for $t \leftarrow 1$ **to** L **do**
  if *deadline reached* **then**
    break;
  $a \leftarrow \text{GREEDYACTION}(\hat{s})$;
  $(\hat{s}, r, \text{done}) \leftarrow \texttt{env.simulate}(\hat{s}, a)$;
  $G \leftarrow G + w\,r$; $w \leftarrow \gamma w$;
  if *done* **then**
    break;
return G;

B4. Empirical investigation of problem complexity

Experimental setup

- Hardware / Python version: **[TODO:]**
- Planning time per step: **[TODO:]**; max steps: **[TODO:]**; α: **[TODO:]**; reward parameters: **[TODO:]**
- Runs per scenario: **[TODO: n]**; random seeds: **[TODO:]**
- 95% CI computed as $\bar{x} \pm t_{0.975,\,n-1}\, s/\sqrt{n}$ **[TODO: (or Wilson interval for success rate)]**.

Performance metrics.

- Success rate (all drones reach goal without collision)
- Total discounted reward
- Number of steps to completion (successful runs)
- Collision rate
- **[TODO: Tree statistics: simulations per step, average tree depth, etc.]**

Set	Varied factor (others fixed)	What we expect to learn
S: State space	**[TODO: grid size, e.g. $5^3, 10^3, 15^3, 20^3$; drones fixed]**	**[TODO: effect of longer horizons / sparser goal reward]**
A: Action space	**[TODO: number of drones N and/or altitude change (8 vs 26 per drone)]**	**[TODO: effect of branching factor on search breadth]**
C: Clutter	**[TODO: number/density of obstacles, narrow passages]**	**[TODO: effect on rollout quality and collision risk]**

Table 3: Experiment design. **[TODO: Note that increasing N also grows $|S|$; explain how the design separates the two factors.]**

Complexity factors and how each is varied (one factor at a time).

ID	Set	Grid	N	Altitude change	#Obstacles	$\lvert A \rvert$
S1	S	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
S2	S	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
S3	S	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A1	A	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A2	A	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A3	A	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C1	C	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C2	C	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C3	C	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

Table 4: Testing scenarios. Config files in `[TODO: experiments/configs/]`.

Scenario list.

Results

ID	Success rate (95% CI)	Discounted reward (mean ± CI)	Steps (mean ± CI)	Sims/step (mean)
S1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
S2	[TODO:]	[TODO:]	[TODO:]	[TODO:]
S3	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A2	[TODO:]	[TODO:]	[TODO:]	[TODO:]
A3	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C2	[TODO:]	[TODO:]	[TODO:]	[TODO:]
C3	[TODO:]	[TODO:]	[TODO:]	[TODO:]

Table 5: Summary statistics over $n =$ [TODO:] runs per scenario.



(a) State space (b) Action space (c) Clutter

Figure 2: Mean discounted reward with 95% CI error bars as each factor increases.

Discussion

Effect of state-space size. [TODO:]

Effect of action-space size. [TODO:]

Effect of clutter. [TODO:]

Which factor degrades performance the most? [TODO: Answer, supported by the statistics (overlapping vs non-overlapping CIs, normalised degradation, etc.).]

Role of each solver component as complexity grows. [TODO: Selection, expansion, rollout/heuristic, backpropagation, tree reuse: which helped or hurt, and why. Ablation results if run.]

Limitations and possible improvements. [TODO:]

Appendix A: Generative AI disclosure

Tools used: [TODO: tool names and versions, or "None"].
What they were used for: [TODO:]
Full list of prompts:

1. [TODO: prompt 1 (verbatim)]
2. [TODO: prompt 2 (verbatim)]

Appendix B: Additional results

[TODO: Optional: per-scenario raw tables, extra plots, ablations.]